SRPO：视觉—语言—动作模型的自参照策略优化

历史论文按新提示词重制版 · CVPR 2026 · 2026-10-05

对应离线阅读包：解压ZIP后双击 index.html。本PDF包含完整13项学习简报，并重点展开创新点、核心算法、数学公式和固定commit源码；完整原文及逐节译文在ZIP中。

1. 基本信息

标题：SRPO: Self-Referential Policy Optimization for Vision-Language-Action Models。作者：Senyu Fei、Siyin Wang、Li Ji、Ao Li、Shiduo Zhang、Liming Liu、Jinlong Hou、Jingjing Gong、Xianzhong Zhao、Xipeng Qiu。机构：复旦大学、同济大学、上海创智学院。

版本：arXiv:2511.15605v2，2025-12-02；CVPR 2026。论文 · CVPR 正式页 · 官方代码 · 文档。

开放资源：siiRL 训练框架与四个 LIBERO 套件启动脚本；V-JEPA 2 权重需另行准备；LIBERO 数据/环境与 OpenVLA 初始化模型需按仓库配置获得。

2. 分类标签与个性化推荐理由

标签：VLA、机器人操作、强化学习、GRPO、世界模型、过程奖励、奖励塑形、OpenVLA、V-JEPA 2、LIBERO。

这篇最能补足你已有 SFT/GRPO 经验中的“长时程信用分配”环节：不是再设计更多规则奖励，而是从当前策略自己的成功 rollout 构造连续进度信号。它也可连接 VGGT/Depth Pro/YOLO：将 V-JEPA 视频潜变量替换或增强为显式 3D 状态，检验几何进度是否能更可靠地区分“看起来接近但物理上失败”的轨迹。

一句话问题定义：机器人执行“把杯子放进碗里”失败时，如何不只给 0 分，而是利用同批成功轨迹判断它已经完成到哪一步，并把该信息用于稳定更新 VLA 策略?

3. 真正创新点：已有方法 vs SRPO

维度	结果奖励 GRPO	人工过程奖励	SRPO
参照	最终 0/1	专家轨迹/任务阶段	当前策略批内成功轨迹
失败信息	组内失败常同分	人工定义里程碑	到成功策略簇中心的潜空间距离
表征	无过程表征	任务专用状态	V-JEPA 2 潜在世界表征
适配成本	低但稀疏	每任务工程高	无需新增专家监督，但需世界模型和成功样本
论文证据	一示范基线 48.9%	TGRPO 平均 80.7%	Online SRPO 99.2%，200 RL 步

创新 1：动态自参照

旧方法使用固定专家数据或终局结果。固定参照跟不上策略演化，二元奖励又浪费失败轨迹。SRPO 每轮从策略自产成功轨迹更新参照；消融显示固定 50 条专家轨迹后期平台，约需 1.4 倍步数仍低于完整方法。沿用部分：策略骨干、环境 rollout、GRPO/PPO clipping 都不是新贡献。

创新 2：潜在世界表征进度

像素距离对背景/视角敏感，ImageBind 不理解机器人动力学。SRPO 借用 V-JEPA 2 视频潜表征，以 L2 距离衡量行为接近成功的程度。700 成功+300 失败轨迹上，SRPO 的 SC/Mono/MMD/JS/SMD 均高于两种对照。

创新 3：成功策略聚类

任务可有多条合法路径；DBSCAN 先形成成功策略簇，再与最近中心比较，避免被单条噪声轨迹误导。去掉聚类前期接近，后期成功模式多样时收敛落后。

4. 方法流程

- 用 OpenVLA* 在 LIBERO 并行 rollout；每个任务/初态采样 G=8 条轨迹。
- 记录完整视频、动作 token、终局成功标志与任务名。
- V-JEPA 2 把每条视频编码成 D 维轨迹嵌入。
- 按任务分组，把成功嵌入用 DBSCAN 聚成 K 个中心。
- 成功轨迹奖励 1；失败轨迹按到最近中心的距离映射成 (0,1) 内过程奖励。
- 组内标准化得到优势，计算新旧策略比率、clip surrogate 与参考模型 KL。
- 只更新 actor；世界模型作为冻结编码器，critic 不启用。

源码 DAG 的真实顺序是 rollout → embodied sampling/filter → reward → advantage → old log-prob → reference log-prob → actor train。永久链接：builtin_pipelines.py:L230-L297。

5. 核心算法一：视频轨迹编码

问题与衔接

把可变长像素序列压成可比较的物理进度表征，输出交给聚类与奖励函数。

输入、输出与形状

设 B 为 rollout 数，T 为每条轨迹帧数，H×W 为图像大小，C=3；输入通常为 B 组 [T,H,W,C] 帧列表。V-JEPA 2 在脚本中使用 384 像素输入，输出嵌入 E∈R^{B×D}；D 由所选 ViT-G 权重决定，代码没有把它写死。

执行顺序

环境生成时收集每个 task_file_name 对应的视频帧；rollout 结束后将任务名和帧列表批量交给 embedding_model.get_embeddings，再转成 float32 TensorDict 字段 vjepa_embedding。编码器推理使用 torch.no_grad 的 rollout 路径，梯度不回到 V-JEPA。

源码：embodied_rollout.py:L456-L471；配置：run...long.sh:L163-L173。

形状例子

B=8，每条取 32 帧，384×384 RGB，概念输入为 [8,32,384,384,3]；编码后若 D=1024，则 E=[8,1024]。后续只保留 E，无需把像素送入奖励函数。

```
# 自行编写的示意伪代码，不是作者原代码
videos = rollout(policy, env, group_size=8)
with no_grad():
E = vjepa.encode(videos) # [B,D]
batch["vjepa_embedding"] = E.float()
```

6. 核心算法二：成功策略聚类与失败进度奖励

输入输出

输入 E=[B,D]、成功标记 y=[B]、任务名 q=[B]；按任务独立处理。输出 g=[B]。成功为 1，合法失败为连续分数，零嵌入为 0。

预处理与聚类

源码先丢弃全零嵌入；成功嵌入标准化后以 DBSCAN(eps=.5,min_samples=2) 聚类，排除噪声标签 −1，簇中心在标准化空间求均值再逆变换。若没有簇，则回退到全部成功嵌入均值。

距离和映射

每个失败嵌入与 K 个中心计算欧氏距离矩阵 [F,K]，取每行最小值。源码在当前任务失败样本内部 min-max 归一化，然后计算 0.6×sigmoid(10×(0.5−d_norm))；因此失败分最高接近 0.596，而不是论文附录 α=0.8 的上限。

数值例子

三个失败到最近中心距离 [0.2,0.5,0.8]，归一为 [0,.5,1]；奖励约 [0.596,0.300,0.004]。若所有距离相同，代码统一设 0.5，奖励 0.3。

```
# 自行编写的示意伪代码
for task in unique(task_name):
C = dbscan_centers(E[(q==task)&success])
d = cdist(E[(q==task)&failure], C).min(axis=1)
z = (d-d.min()) / max(d.max()-d.min(), 1e-6)
g[failure] = 0.6 * sigmoid(10*(0.5-z))
g[success] = 1.0
```

源码：聚类 L47-L68；奖励 L186-L256。

7. 核心算法三：SRPO/GRPO 策略更新

张量与序列

B 为 prompt 数，G=8 为每 prompt rollout 数，L 为动作响应 token 数（脚本 response 上限 512，动作每步 7 token、chunk 8）。log-prob 张量概念上为 [B×G,L]，response mask 同形状；轨迹奖励 [B×G] 广播到有效动作 token。

训练

组内奖励标准化为 Â；新旧 log-prob 差指数化得 r；正优势时 ratio 增大会提高目标，负优势时 ratio 降低；clip 把一次更新限制在 $[1-\varepsilon_{low}, 1+\varepsilon_{high}]$。参考策略 KL 抑制偏离。仅 actor 参数更新；V-JEPA 与 reference 冻结，源码显式关闭 critic。

推理

部署只保留训练后的 VLA 策略，不再需要组采样、DBSCAN、参考策略或优势计算。它仍按 7 维动作 token、8 步 action chunk 执行。

数值例子

若旧概率 .20、新概率 .24，则 r=1.2。Â=0.5、ε=0.2 时 unclipped 与 clipped 都为 0.6；新概率升到 .30 时 r=1.5，正优势目标仍被截到 1.2×0.5=0.6，防止过大更新。

```
# 自行编写的示意伪代码
A = (g - group_mean(g)) / (group_std(g) + 1e-6)
ratio = exp(logp_new - logp_old)
surrogate = min(ratio*A, clip(ratio, 1-eps_lo, 1+eps_hi)*A)
loss = -masked_mean(surrogate) + beta*KL(policy, reference)
loss.backward(); actor_optimizer.step()
```

脚本配置：L48-L64、L118-L147、L184-L199。

8. 数学公式详解

8.1 潜表征与距离（式 2–4）

$h_i = W(o^{(i)}_{0:T})$；$d_i = \min_{h_j \in C} \| h_i - h_j \|^2$

W 把 [T,H,W,C] 视频变为 [D]；C=[K,D]。广播相减得到 [K,D]，沿 D 求平方和得到 [K]，再沿 K 取最小值。使用平方 L2 是为了让离成功模式更远的差异快速增大；余弦距离会忽略幅值，像素 L1 则受视觉扰动更强。代码实际 cdist(...,"euclidean") 没有平方，这是论文—实现差异。

8.2 奖励映射（式 5）

$g_i = 1$（成功）；$\phi((d_i - \bar{d})/\sigma_d)$（失败）

z-score 消除不同批次距离尺度，ϕ 把无界值压到 (0,1)。要满足“距离越小奖励越大”，ϕ 应是单调递减形式，例如 sigmoid(−z)；论文只写 activation，符号未完全明确。代码改为任务内 min-max 加反向 sigmoid，且上限乘 0.6。

8.3 组优势（式 6、9）

$\hat{A}_i = (g_i - \mu_g)/\sigma_g$

同组相对标准化不需要 critic。例：g=[1,.6,.2]，均值 .6，标准差约 .327，则优势约 [1.225,0,−1.225]。梯度推动第一条动作概率上升，第三条下降。组内全同分时标准差接近 0，仍缺学习信号；SRPO 的连续失败分减少这种情况。

8.4 PPO clip（式 7）

$L^{CLIP} = \min(r\hat{A}, \text{clip}(r, 1-\varepsilon, 1+\varepsilon)\hat{A})$

r 是逐 token 新旧策略概率比，[B×G,L]；A 是轨迹标量，广播后用 response mask 聚合。min 形成保守下界；与不截断 policy gradient 相比更稳。非对称 clip 在公开脚本中为 low=.2、high=.28。

8.5 总目标与 KL（式 8）

$L_{SRPO} = E[L^{CLIP}] + \beta D_{KL}(\pi_\theta \| \pi_{ref})$

若“最大化”，KL 通常应以负号出现；若代码最小化 loss，则常写 −surrogate+βKL。论文式 (8) 把“最大化”和“+KL”并列，存在符号歧义，不能自行当成作者推导。工程实现应以训练框架实际 loss 符号为准。

9. 论文与公开实现的差异、复现注意点

项目	论文	commit 89d876… 源码
距离	平方 L2	scipy cdist 欧氏距离
归一化	失败距离 z-score	每任务失败样本 min-max
失败奖励上限	附录 α 最优 0.8	硬编码 0.6×sigmoid
DBSCAN	公式写 DBSCAN(S)	标准化成功嵌入，eps=.5、min_samples=2；无簇回退均值
动作模型	OpenVLA* 离散动作	脚本变量称 OpenVLA-OFT，但仍配置动作 token

另有配置不一致：论文附录写 prompt 512、response 128；公开 run...long.sh 变量写 prompt 256、response 128，但命令又把 rollout response_length 设为 512。复现时必须记录最终 Hydra 配置，而不能只抄论文。

奖励要求同任务批内同时有成功和失败；源码若某任务 success_count=0 或 fail_count=0，就保留初始 0 奖励。因此前期策略完全不会成功时，SRPO 本身无法凭空产生参照。公开脚本用成功率 [0.1,0.9] 的 group filter 缓解全同分组。

10. 实验与证据

- LIBERO：一示范基线 48.9%，Offline SRPO 92.5%，Online SRPO 99.2%；相对基线是 50.3 个百分点、103% 相对提升。
- LIBERO-Plus：零样本总分 19.4→59.6；有扰动训练 30.7→82.1。论文所谓 167% 是特定口径的相对提升，不等于 167 个百分点。
- 奖励表征：SC .987、Mono .978、MMD .589、JS .557、SMD 15.637，高于像素与 ImageBind。
- 消融：固定外部参照后期平台；去聚类后期收敛变慢；α=.8 最优。

注意：表 1 中 SRPO 平均 99.2 高于 RLinf 98.0，但 Spatial 单项 RLinf 99.4 高于 SRPO 98.8；不能写成“每个指标全面领先”。

11. 局限与适用范围

作者报告/材料可见：真实机器人只做离线 RL；V-JEPA 2、OpenVLA、LIBERO 和并行 rollout 带来明显算力/工程成本；批内无成功轨迹时无法形成自参照。本简报推断：潜空间的最近中心可能把外观相似但接触状态错误的轨迹打高分；DBSCAN 对 eps、样本量与尺度敏感；动态自参照若成功样本本身走次优捷径，可能强化捷径。

适合：可自动验证终局成功、有视频 rollout、能在一个任务内采到至少少量成功的机器人操作。暂不宜直接用于：安全代价极高的真实在线探索、成功极罕见的冷启动任务、只看单帧无法判断进度的部分可观测任务。

12. 前置知识与 45 分钟学习路线

前置知识

- GRPO/PPO clipping：理解组优势与概率比怎样限制策略步长。
- 奖励塑形：过程信号能加速学习，但错误 shaping 会改变最优策略。
- 视频世界模型：潜表征不仅编码外观，还需表达对象交互与时序进度。
- DBSCAN：无需预设簇数，可把离群成功轨迹标为噪声。
- 动作分块：一次预测多步动作以降低推理延迟，但训练 log-prob 的 mask/长度更复杂。

45 分钟

- 0–8 分钟：看图 1/2，复述“成功参照→潜表征→失败进度→优势”。
- 8–20 分钟：精读方法 3.2–3.3，手算 [1,.6,.2] 的组优势。
- 20–30 分钟：对照 embodied.py:L47-L68,L186-L256，找出平方 L2、z-score、α=.8 与代码差异。
- 30–38 分钟：看表 1、LIBERO-Plus 表与消融，区分百分点和相对提升。
- 38–45 分钟：设计“显式 3D + V-JEPA”奖励对照。

必须看懂：式 (5)
为什么要让失败轨迹产生连续且低于成功的奖励。思考题：同一任务存在两条视觉完全不同的合法成功路径时，DBSCAN 簇中心是否足够？若每个簇只有 1 条样本，如何稳定估计进度？

13. 代码学习任务、面试题与迁移实验

最值得读的源码

- embodied.py:L47-L68 与 L186-L256：聚类、距离、归一化、sigmoid。
- embodied_rollout.py:L435-L471：成功标记、任务名、V-JEPA 嵌入如何装入 TensorDict。
- run...long.sh:L30-L69 与 L101-L199：动作、采样、clip、环境和资源配置。

最小路线：先只运行奖励单元测试：人为构造 2 成功+3 失败的 [5,D] 嵌入，调用 _compute_all_rewards，检查成功=1、近中心失败分更高、无成功时分数为 0。再运行 LIBERO 单任务小批 rollout。完整论文训练为 8×A100 SFT；RL 公开脚本默认 8 GPU/节点。单次奖励函数 CPU 验证不需要这些 GPU，不能把完整训练资源等同于推理需求。

面试问题

Q1：SRPO 与 GRPO 的关键差异？答：优化骨架仍是组相对优势与 clip；差异在奖励，SRPO 用批内成功轨迹的世界模型潜表示给失败轨迹连续进度，缓解同组失败全 0。

Q2：为何聚类成功轨迹，而不是找最近单条成功轨迹？答：多策略任务需要多个模式；簇中心降低单条成功中的噪声段影响。消融显示多样性增加后，去聚类版本后期落后。

迁移实验：几何增强 SRPO

改动位置：在 embodied_rollout.py:L456-L471 同步提取 VGGT/Depth Pro 的相机位姿、深度和点图统计，将 [V-JEPA embedding; 几何 embedding] 拼接或分别算距离；在 embodied.py:L232-L252 把距离改成 $\lambda d_{world}+(1-\lambda)d_{geo}$。

预期收益：减少“外观接近、几何接触错误”的假高分，尤其适合遮挡、目标移动与抓取失败。待验证假设：显式几何进度与任务成功单调相关，且 VGGT/Depth Pro 在机器人近距离视角足够稳定。

最小对照：固定同一 OpenVLA checkpoint 和 10 个 LIBERO-Object 任务，比较 A:终局 0/1，B:V-JEPA SRPO，C:几何 SRPO，D:融合 SRPO；报告 50/100/200 步成功率、奖励 SC/Mono、失败分布 AUC 和额外延迟。对 YOLO 检出的目标只用于几何 ROI，不把类别置信度直接当奖励。对未来 Qwen2.5-VL/VLA Agent，可把高质量同批成功轨迹作为自参照，但需保留可验证终局标准，避免语义相似度自我循环。